# Universidad Tecnológica de Durango

## Tecnologías de la Información

### Proyecto Integrador I

Informe Técnico de Unidad III · A08

*“Diseño funcional: algoritmos, diagramas de flujo y casos de uso”*


Alumnos:

- Urbina Avitia Rodolfo
- Guzmán Reyes Diego Emmanuel
- Navar Díaz Gabriel de Jesús
- Ortega Alvarado Luis Francisco

4°A

Docente:

- M.A. Raúl Iván Herrera González


Noviembre 2026

# Tabla de ilustraciones

Ilustración 1 Diagrama de casos de uso del Sistema de Gestión de Clientes Mqueen ..... 5
Ilustración 2 Diagrama de flujo de ALG-01 Registrar refacción ..... 17
Ilustración 3 Diagrama de flujo de ALG-02 Descontar refacciones de una orden ..... 18
Ilustración 4 Diagrama de flujo de ALG-03 Generar lista de alertas ..... 18
Ilustración 5 Diagrama de flujo de ALG-04 Iniciar sesión ..... 19

# Tabla de contenido

Propósitos ..... 4
General ..... 4
Específicos ..... 4
Actividades del reporte ..... 5
Diagrama de casos de uso ..... 5
Especificación de casos de uso ..... 6
CU-01 Iniciar sesión ..... 6
CU-02 Gestionar refacciones ..... 7
CU-03 Registrar orden de servicio y descontar refacciones ..... 7
CU-06 Consultar alertas de stock mínimo ..... 8
CU-08 Consultar historial de servicios ..... 8
Reglas de negocio ..... 9
Algoritmos ..... 10
ALG-01 Registrar refacción ..... 10
ALG-02 Descontar refacciones de una orden de servicio ..... 10
ALG-03 Generar lista de alertas de stock mínimo ..... 11
ALG-04 Iniciar sesión ..... 12
Pruebas de escritorio ..... 13
Prueba de ALG-01 Registrar refacción ..... 13
Prueba de ALG-02 Descontar refacciones ..... 14
Prueba de ALG-03 Generar lista de alertas ..... 16
Prueba de ALG-04 Iniciar sesión ..... 16
Diagramas de flujo ..... 18
Módulos y principios de diseño funcional ..... 20
Matriz de trazabilidad ..... 21
Revisión entre pares ..... 22
Conclusiones ..... 23
Declaración de uso de IA ..... 23
Bibliografía ..... 24

# Propósitos

## *General*

- Diseñar la lógica de negocio del Sistema de Gestión de Clientes Mqueen mediante casos de uso, algoritmos en pseudocódigo y diagramas de flujo, a partir de las historias de usuario de A05, para que sirvan de guía directa del código en los sprints de desarrollo.

## *Específicos*

- Identificar los actores y casos de uso del sistema y representarlos en un diagrama de casos de uso UML con límite del sistema y relaciones <<include>> y <<extend>>.
- Especificar los casos de uso prioritarios ligados a las historias Must (HU-01 y HU-02), con flujo básico, flujos alternos y postcondiciones.
- Catalogar las reglas de negocio (validaciones, cálculos, restricciones y permisos) que aparecen en los casos de uso.
- Escribir algoritmos en pseudocódigo con estructuras de decisión y repetición, comprobarlos con pruebas de escritorio y representarlos en diagramas de flujo ANSI/ISO 5807.
- Proponer los módulos del sistema aplicando modularidad, alta cohesión, bajo acoplamiento y separación de responsabilidades, y conectar cada requisito con su diseño en una matriz de trazabilidad.

# Actividades del reporte

## *Diagrama de casos de uso*

El diagrama se construyó a partir de las cuatro historias de usuario de A05 (HU-01 a HU-04) y del requisito de acceso con usuario y contraseña (RNF-02). Por indicación del docente, el sistema tiene un solo usuario, el Administrador, que corresponde a la encargada de administración y almacén. Los mecánicos y los clientes no tienen cuenta; los mecánicos reportan a la encargada las refacciones que usan y ella las captura en la orden de servicio. Por eso hay un solo actor.

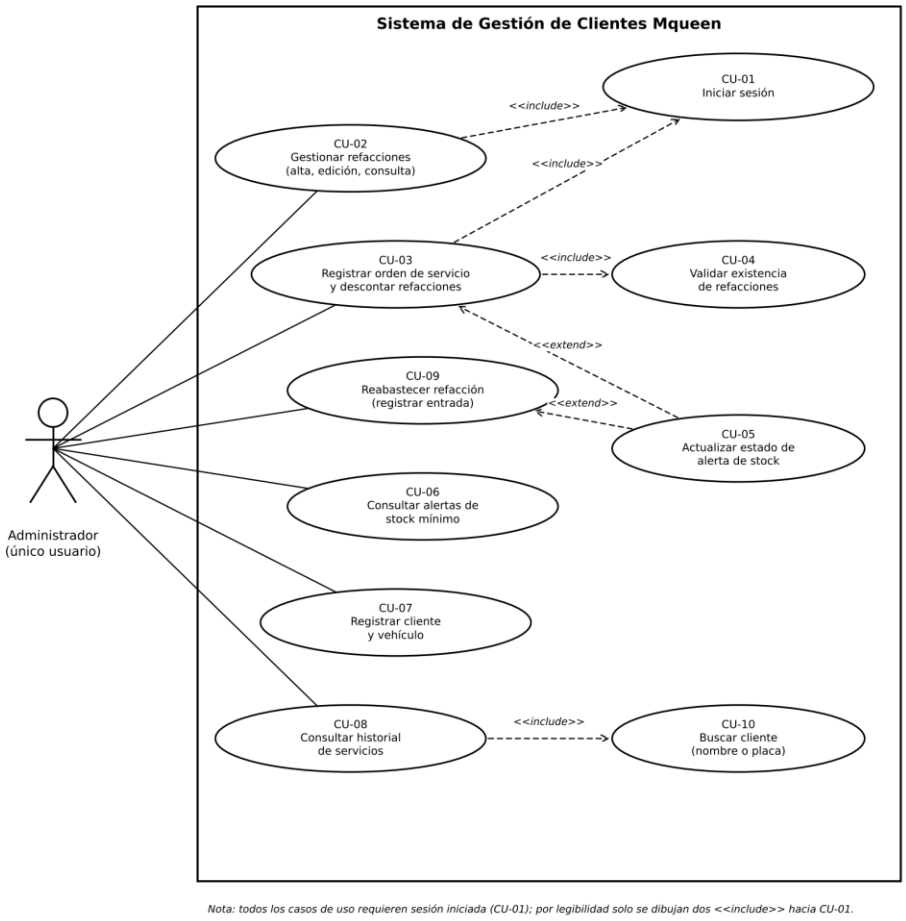



*Ilustración 1 Diagrama de casos de uso del Sistema de Gestión de Clientes Mqueen*

La siguiente tabla describe a cada actor y los casos de uso en los que participa.

| Actor | Descripción | Casos de uso en los que participa |
|---|---|---|

| Actor | Descripción | Casos de uso en los que participa |
|---|---|---|
| **Administrador (encargada de administración y almacén)** | Único usuario del sistema. Controla el catálogo de refacciones, el reabastecimiento y las alertas; registra a los clientes y sus vehículos; y captura en cada orden de servicio las refacciones que reportan los mecánicos. | CU-01, CU-02, CU-03, CU-06, CU-07, CU-08, CU-09 (y CU-04, CU-05 y CU-10 por inclusión o extensión) |

Relaciones del diagrama: todos los casos de uso incluyen CU-01 Iniciar sesión; CU-03 incluye CU-04 Validar existencia, porque nunca se descuenta sin validar; CU-05 Actualizar estado de alerta extiende a CU-03 y a CU-09, porque solo ocurre cuando el stock cruza el stock mínimo; y CU-08 incluye CU-10 Buscar cliente.

## *Especificación de casos de uso*

Se especifican cinco casos de uso: los dos ligados a las historias Must (CU-02 y CU-03), CU-01 porque todos los demás lo incluyen, y CU-06 y CU-08, que atienden las historias Should del Sprint 2.

## *CU-01 Iniciar sesión*

| Campo | Contenido |
|---|---|
| **ID y nombre** | CU-01 Iniciar sesión |
| **Actor(es)** | Administrador |
| **Requisito / historia** | RNF-02 / sin historia propia (se verifica en la DoD de todas las historias) |
| **Precondiciones** | Existe la cuenta del administrador en la BD y su contraseña está guardada con hash. |
| **Flujo básico** | 1. El usuario abre el sistema en el navegador.<br>2. El sistema muestra el formulario de usuario y contraseña.<br>3. El usuario captura sus datos y presiona «Entrar».<br>4. El sistema busca al usuario y compara el hash de la contraseña.<br>5. El sistema abre la sesión y muestra el panel principal. |
| **Flujos alternos / de error** | 4a. Usuario o contraseña incorrectos: el sistema muestra «Usuario o contraseña incorrectos» sin indicar cuál dato falló y suma un intento; regresa al paso 2.<br>4b. Tercer intento fallido: el sistema bloquea la cuenta 5 minutos y lo informa. |
| **Postcondiciones** | Existe una sesión activa del administrador; sin sesión no se puede abrir ninguna otra pantalla (RN-07). |
| **Reglas de negocio** | RN-07, RN-09 |

## CU-02 Gestionar refacciones

| Campo | Contenido |
|---|---|
| **ID y nombre** | CU-02 Gestionar refacciones (alta, edición y consulta) |
| **Actor(es)** | Administrador |
| **Requisito / historia** | RF-01 / HU-01 (Must) |
| **Precondiciones** | El administrador inició sesión (CU-01). |
| **Flujo básico** | 1. El administrador entra a «Refacciones» y presiona «Nueva refacción».<br>2. El sistema muestra el formulario: clave, nombre, stock, precio y stock mínimo.<br>3. El administrador captura los datos y presiona «Guardar».<br>4. El sistema valida campos obligatorios, valores numéricos y clave única.<br>5. El sistema guarda la refacción, revisa si debe marcarse en alerta y la muestra en la lista. |
| **Flujos alternos / de error** | 4a. Falta un campo obligatorio: el sistema indica qué campo falta y no modifica la BD (criterio 2 de HU-01).<br>4b. Stock o stock mínimo negativos, o precio menor o igual a 0: el sistema muestra «Cantidades o precio no válidos».<br>4c. La clave ya existe: el sistema muestra «La clave ya existe» y no guarda.<br>1a. Edición: el administrador busca la refacción, presiona «Editar», modifica los datos y continúa en el paso 3 (la clave no se puede cambiar). |
| **Postcondiciones** | La refacción queda guardada o actualizada en la BD y aparece en la lista; si su stock es menor o igual al mínimo queda marcada en alerta. |
| **Reglas de negocio** | RN-01, RN-02, RN-03, RN-06 |

## CU-03 Registrar orden de servicio y descontar refacciones

| Campo | Contenido |
|---|---|
| **ID y nombre** | CU-03 Registrar orden de servicio y descontar refacciones |
| **Actor(es)** | Administrador (captura lo que reporta el mecánico) |
| **Requisito / historia** | RF-02 / HU-02 (Must) |
| **Precondiciones** | El administrador inició sesión; existe la orden de servicio del vehículo, ligada a un cliente registrado (RN-12); las refacciones están dadas de alta (CU-02); el mecánico reportó las piezas que usó (RN-08). |
| **Flujo básico** | 1. El administrador abre la orden de servicio del vehículo.<br>2. Presiona «Agregar refacción», la busca por clave o nombre y captura la cantidad que reportó el mecánico; repite por cada refacción.<br>3. Presiona «Confirmar salida».<br>4. El sistema agrupa las partidas repetidas y valida la existencia de todas (incluye CU-04).<br>5. El sistema descuenta cada cantidad del stock y registra la salida ligada a la orden, todo en una sola transacción.<br>6. Si alguna refacción queda en su stock mínimo o por debajo, se marca en alerta (extiende CU-05). |

| Campo | Contenido |
|---|---|
| | 7. El sistema muestra «Salida registrada» y el nuevo stock. |
| **Flujos alternos / de error** | 4a. Una cantidad pedida es mayor que el stock: el sistema avisa «No hay existencia suficiente» con la cantidad disponible y no guarda ninguna partida (criterio 2 de HU-02).<br>4b. Una cantidad es 0 o negativa: el sistema muestra «Cantidad no válida».<br>3a. La orden no tiene partidas: el sistema pide agregar al menos una refacción.<br>5a. Falla la conexión con la BD a mitad del descuento: la transacción se revierte y el stock queda como estaba. |
| **Postcondiciones** | El stock de cada refacción disminuyó exactamente en la cantidad usada (p. ej., 2 filtros → stock − 2) y la salida queda en el historial de la orden. |
| **Reglas de negocio** | RN-04, RN-05, RN-06, RN-08, RN-12 |

## *CU-06 Consultar alertas de stock mínimo*

| Campo | Contenido |
|---|---|
| **ID y nombre** | CU-06 Consultar alertas de stock mínimo |
| **Actor(es)** | Administrador |
| **Requisito / historia** | RF-03 / HU-03 (Should) |
| **Precondiciones** | El administrador inició sesión; hay refacciones registradas. |
| **Flujo básico** | 1. El administrador entra al panel principal.<br>2. El sistema recorre las refacciones y arma la lista de las que tienen stock menor o igual a su mínimo.<br>3. El sistema muestra la lista destacada en rojo, ordenada de menor a mayor stock. |
| **Flujos alternos / de error** | 2a. Ninguna refacción está en su mínimo: el sistema muestra «No hay refacciones en stock mínimo».<br>3a. El administrador reabastece una refacción (CU-09) por encima del mínimo: al actualizar la BD la refacción deja de aparecer en la lista (criterio 2 de HU-03). |
| **Postcondiciones** | El administrador conoce qué refacciones debe pedir a los proveedores. |
| **Reglas de negocio** | RN-06 |

## *CU-08 Consultar historial de servicios*

| Campo | Contenido |
|---|---|
| **ID y nombre** | CU-08 Consultar historial de servicios |
| **Actor(es)** | Administrador |
| **Requisito / historia** | RF-04 / HU-04 (Should) |
| **Precondiciones** | El administrador inició sesión. |
| **Flujo básico** | 1. El administrador escribe el nombre del cliente o la placa del vehículo (incluye CU- |

| Campo | Contenido |
|---|---|
| | 10).<br>2. El sistema muestra las coincidencias.<br>3. El administrador selecciona el perfil del cliente.<br>4. El sistema muestra, en orden cronológico, sus servicios y las refacciones usadas en cada uno. |
| **Flujos alternos / de error** | 2a. No hay coincidencias: el sistema ofrece el botón «Nuevo cliente» (CU-07, criterio 2 de HU-04).<br>4a. El cliente no tiene servicios: el sistema muestra «Sin servicios registrados». |
| **Postcondiciones** | El administrador conoce las piezas cambiadas antes al vehículo y se las comunica al mecánico para un mejor diagnóstico. |
| **Reglas de negocio** | RN-10, RN-11 |

## *Reglas de negocio*

El catálogo reúne las reglas que aparecen en los casos de uso. Las reglas RN-09 (bloqueo tras 3 intentos) y RN-12 son propuestas del equipo para este diseño y se validarán con la encargada del taller.

| ID | Regla | Caso de uso | Tipo |
|---|---|---|---|
| RN-01 | La clave de cada refacción es única y no se puede modificar después del alta. | CU-02 | Validación |
| RN-02 | Clave, nombre, stock, precio y stock mínimo son obligatorios. | CU-02 | Validación |
| RN-03 | Stock y stock mínimo son enteros mayores o iguales a 0; el precio es mayor que 0. | CU-02, CU-09 | Validación |
| RN-04 | No se puede descontar más piezas de las que hay en stock; si una partida no cumple, no se guarda ninguna (todo o nada). | CU-03, CU-04 | Restricción |
| RN-05 | Nuevo stock = stock actual − cantidad usada (salida); nuevo stock = stock actual + cantidad recibida (entrada). | CU-03, CU-09 | Cálculo |
| RN-06 | Una refacción está en alerta cuando su stock es menor o igual a su stock mínimo. | CU-05, CU-06 | Cálculo |
| RN-07 | El sistema tiene un solo usuario (administrador); todas las pantallas requieren su sesión activa y no existen cuentas para mecánicos ni clientes. | Todos (CU-01) | Permiso |
| RN-08 | Los mecánicos no usan el sistema: reportan las piezas usadas al administrador, quien las captura en la orden de servicio. | CU-03 | Restricción |
| RN-09 | Las contraseñas se guardan con hash; tras 3 intentos fallidos la cuenta se bloquea 5 minutos. | CU-01 | Restricción |
| RN-10 | La placa de cada vehículo es única y el teléfono del cliente tiene 10 dígitos. | CU-07, CU-10 | Validación |
| RN-11 | El historial de un cliente se muestra en orden cronológico. | CU-08 | Restricción |

| ID | Regla | Caso de uso | Tipo |
|---|---|---|---|
| RN-12 | Toda orden de servicio está ligada a un cliente y a un vehículo registrados. | CU-03 | Restricción |

## Algoritmos

Se escribieron cuatro algoritmos de la lógica de negocio en pseudocódigo. Cada uno usa estructuras de decisión (Si…Entonces) y de repetición (Para o Mientras) e indica entre comentarios la regla de negocio que aplica.

## ALG-01 Registrar refacción

| | |
|---|---|
| **Propósito** | Validar y guardar una refacción nueva en el catálogo (CU-02, HU-01). |
| **Entradas** | clave, nombre, stock, precio, stockMinimo (con sesión activa, RN-07). |
| **Salidas** | Refacción guardada y mensaje de confirmación, o mensaje de error sin modificar la BD. |

```
Inicio RegistrarRefaccion(clave, nombre, stock, precio, stockMinimo)
  campos <- [clave, nombre, stock, precio, stockMinimo]
  errores <- 0
  Para i <- 1 Hasta 5 Hacer                          // RN-02
    Si campos[i] está vacío Entonces
      Mostrar "Falta el campo " + nombreCampo(i)
      errores <- errores + 1
    FinSi
  FinPara
  Si errores > 0 Entonces Terminar FinSi
  Si stock < 0 O stockMinimo < 0 O precio <= 0 Entonces   // RN-03
    Mostrar "Cantidades o precio no válidos"; Terminar
  FinSi
  Si existeClave(clave) Entonces                     // RN-01
    Mostrar "La clave ya existe"; Terminar
  FinSi
  guardarRefaccion(clave, nombre, stock, precio, stockMinimo)
  Si stock <= stockMinimo Entonces                   // RN-06
    marcarAlerta(clave)
  FinSi
  Mostrar "Refacción registrada"
Fin
```

## ALG-02 Descontar refacciones de una orden de servicio

| | |
|---|---|
| **Propósito** | Descontar del inventario las refacciones usadas en una orden, sin permitir stock negativo (CU-03, CU-04, HU-02). |
| **Entradas** | idOrden; lista de partidas (clave, cantidad). |
| **Salidas** | Stock actualizado, salidas registradas y alertas marcadas, o mensaje de existencia insuficiente sin cambios en la BD. |

```
Inicio DescontarRefacciones(idOrden, partidas)
  // partidas: pares (clave, cantidad) que reportó el mecánico (RN-08)
  Si NO existeOrden(idOrden) Entonces                  // RN-12
    Mostrar "La orden no existe"; Terminar
  FinSi
  n <- longitud(partidas)
  Si n = 0 Entonces
    Mostrar "Agrega al menos una refacción"; Terminar
  FinSi
  partidas <- agruparPorClave(partidas)  // suma cantidades si una clave se repite
  n <- longitud(partidas)
  // Fase 1: validar TODAS las partidas antes de modificar la BD (RN-04)
  i <- 1
  Mientras i <= n Hacer
    disponible <- stockActual(partidas[i].clave)
    Si partidas[i].cantidad <= 0 Entonces
      Mostrar "Cantidad no válida"; Terminar
    FinSi
    Si partidas[i].cantidad > disponible Entonces
      Mostrar "No hay existencia suficiente de " + partidas[i].clave
              + " (disponible: " + disponible + ")"
      Terminar                                         // no se guarda nada
    FinSi
    i <- i + 1
  FinMientras
  // Fase 2: descontar dentro de una transacción (RN-05)
  iniciarTransaccion()
  Para i <- 1 Hasta n Hacer
    nuevoStock <- stockActual(partidas[i].clave) - partidas[i].cantidad
    actualizarStock(partidas[i].clave, nuevoStock)
    registrarSalida(idOrden, partidas[i].clave, partidas[i].cantidad)
    Si nuevoStock <= stockMinimo(partidas[i].clave) Entonces   // RN-06
      marcarAlerta(partidas[i].clave)
    FinSi
  FinPara
  confirmarTransaccion()
  Mostrar "Salida registrada"
Fin
```

Se validan todas las partidas antes de descontar y se usa una transacción para que la salida sea «todo o nada»: si se capturan 2 filtros y 5 balatas y solo hay 3 balatas, no se descuentan los filtros. La agrupación por clave evita que dos partidas de la misma refacción pasen la validación por separado y juntas dejen el stock en negativo.

## *ALG-03 Generar lista de alertas de stock mínimo*

| | |
|---|---|
| **Propósito** | Obtener las refacciones en su stock mínimo o por debajo para destacarlas en el panel (CU-05, CU-06, HU-03). |
| **Entradas** | Ninguna (lee el catálogo de refacciones de la BD). |
| **Salidas** | Lista ordenada de refacciones en alerta, o mensaje de que no hay alertas. |

```
Inicio GenerarListaAlertas()
  lista <- vacía
  refacciones <- obtenerRefacciones()
  n <- longitud(refacciones)
  Para i <- 1 Hasta n Hacer
    Si refacciones[i].stock <= refacciones[i].stockMinimo Entonces   // RN-06
      agregar(lista, refacciones[i])
      refacciones[i].alerta <- Verdadero
    SiNo
      refacciones[i].alerta <- Falso
    FinSi
  FinPara
  Si lista está vacía Entonces
    Mostrar "No hay refacciones en stock mínimo"
  SiNo
    ordenar(lista) por stock ascendente
    Mostrar lista destacada en rojo en el panel
  FinSi
Fin
```

## *ALG-04 Iniciar sesión*

| | |
|---|---|
| **Propósito** | Autenticar al administrador con hash de contraseña, limitar los intentos y abrir la sesión (CU-01, RNF-02). |
| **Entradas** | usuario y contraseña (hasta 3 veces). |
| **Salidas** | Sesión abierta con el panel principal, o cuenta bloqueada 5 minutos. |

```
Inicio IniciarSesion()
  intentos <- 0
  acceso <- Falso
  Mientras intentos < 3 Y acceso = Falso Hacer        // RN-09
    Leer usuario, contraseña
    registro <- buscarUsuario(usuario)
    Si registro existe Y verificarHash(contraseña, registro.hash) Entonces
      acceso <- Verdadero
    SiNo
      intentos <- intentos + 1
      Mostrar "Usuario o contraseña incorrectos"
    FinSi
  FinMientras
  Si acceso = Verdadero Entonces                     // RN-07
    abrirSesion(usuario)
    Mostrar panel principal
  SiNo
    bloquear(usuario, 5 minutos)
    Mostrar "Cuenta bloqueada 5 minutos"
  FinSi
Fin
```

## Pruebas de escritorio

Cada algoritmo se ejecutó a mano con dos conjuntos de datos: uno válido y uno que activa un error. En cada traza se registra, paso por paso, el valor de las variables, la condición evaluada y la salida.

## Prueba de ALG-01 Registrar refacción

| Conjunto | Datos de entrada | Resultado esperado |
|---|---|---|
| 1 (válido) | Sesión activa; clave = FIL-001; nombre = Filtro de aceite; stock = 12; precio = 95.00; stockMinimo = 4; FIL-001 no existe en la BD | Se guarda la refacción, no se marca alerta (12 > 4) y se muestra «Refacción registrada». |
| 2 (error) | Sesión activa; clave = BAL-010; nombre = Balata delantera; stock = (vacío); precio = 480.00; stockMinimo = 2 | Se muestra «Falta el campo stock», errores = 1 y la BD no se modifica (criterio 2 de HU-01). |

**Traza del conjunto 1 (válido):**

| Paso | i | errores | Condición evaluada | Salida |
|---|---|---|---|---|
| 1 | — | 0 | errores <- 0; i <- 1 | — |
| 2 | 1 | 0 | 1 <= 5: Verdadero; campos[1] (clave) está vacío: Falso | — |
| 3 | 2 | 0 | 2 <= 5: Verdadero; campos[2] (nombre) está vacío: Falso | — |
| 4 | 3 | 0 | 3 <= 5: Verdadero; campos[3] (stock) está vacío: Falso | — |
| 5 | 4 | 0 | 4 <= 5: Verdadero; campos[4] (precio) está vacío: Falso | — |
| 6 | 5 | 0 | 5 <= 5: Verdadero; campos[5] (stockMinimo) está vacío: Falso | — |
| 7 | 6 | 0 | 6 <= 5: Falso (sale del ciclo); errores > 0: Falso | — |
| 8 | — | 0 | 12 < 0 o 4 < 0 o 95.00 <= 0: Falso | — |
| 9 | — | 0 | existeClave(FIL-001): Falso | Se guarda FIL-001. |
| 10 | — | 0 | 12 <= 4: Falso (no se marca alerta) | Refacción registrada |

**Traza del conjunto 2 (error):**

| Paso | i | errores | Condición evaluada | Salida |
|---|---|---|---|---|
| 1 | — | 0 | errores <- 0; i <- 1 | — |
| 2 | 1 | 0 | 1 <= 5: Verdadero; campos[1] (clave) está vacío: Falso | — |
| 3 | 2 | 0 | 2 <= 5: Verdadero; campos[2] (nombre) está vacío: Falso | — |
| 4 | 3 | 1 | 3 <= 5: Verdadero; campos[3] (stock) está vacío: Verdadero; errores <- 1 | Falta el campo stock |
| 5 | 4 | 1 | 4 <= 5: Verdadero; campos[4] (precio) está vacío: Falso | — |
| 6 | 5 | 1 | 5 <= 5: Verdadero; campos[5] (stockMinimo) está vacío: Falso | — |
| 7 | 6 | 1 | 6 <= 5: Falso (sale del ciclo); errores > 0: Verdadero | Termina; la BD no se modifica. |

## *Prueba de ALG-02 Descontar refacciones*

| Conjunto | Datos de entrada | Resultado esperado |
|---|---|---|
| 1 (válido) | idOrden = OS-015 (existe); partidas = [(FIL-001, 2)]; stock FIL-001 = 12; stockMinimo = 4 | Stock de FIL-001 queda en 10, se registra la salida, no hay alerta y se muestra «Salida registrada» (criterio 1 de HU-02). |
| 2 (error) | idOrden = OS-016 (existe); partidas = [(FIL-001, 2), (BAL-010, 5)]; stock FIL-001 = 10; stock BAL-010 = 3 | En i = 2 la condición 5 > 3 es verdadera: se muestra «No hay existencia suficiente de BAL-010 (disponible: 3)» y no se descuenta ninguna partida (criterio 2 de HU-02). |

**Traza del conjunto 1 (válido):**

| Paso | i | clave | cantidad | disponible / nuevoStock | Condición evaluada | Salida |
|---|---|---|---|---|---|---|
| 1 | — | — | — | — | existeOrden(OS-015): Verdadero; n <- 1 | — |
| 2 | — | — | — | — | n = 0: Falso; agruparPorClave: n <- 1; i <- 1 | — |

| Paso | i | clave | cantidad | disponible / nuevoStock | Condición evaluada | Salida |
|---|---|---|---|---|---|---|
| 3 | 1 | FIL-001 | 2 | 12 | 1 <= 1: Verdadero; 2 <= 0: Falso; 2 > 12: Falso | — |
| 4 | 2 | — | — | — | 2 <= 1: Falso (sale de la validación); iniciarTransaccion(); i <- 1 | — |
| 5 | 1 | FIL-001 | 2 | 10 | nuevoStock <- 12 - 2 = 10; 10 <= 4: Falso | Actualiza el stock y registra la salida. |
| 6 | 2 | — | — | — | 2 <= 1: Falso (sale del ciclo); confirmarTransaccion() | Salida registrada |

**Traza del conjunto 2 (error):**

| Paso | i | clave | cantidad | disponible / nuevoStock | Condición evaluada | Salida |
|---|---|---|---|---|---|---|
| 1 | — | — | — | — | existeOrden(OS-016): Verdadero; n = 2 | — |
| 2 | — | — | — | — | n = 0: Falso; se validan todas las partidas | — |
| 3 | 1 | FIL-001 | 2 | 10 | 2 <= 0: Falso; 2 > 10: Falso | — |
| 4 | 2 | BAL-010 | 5 | 3 | 5 <= 0: Falso; 5 > 3: Verdadero | No hay existencia suficiente de BAL-010 (disponible: 3). |
| 5 | — | — | — | — | Termina antes de iniciar la transacción | No se descuenta ninguna partida. |
| 6 | — | — | — | — | — | BD sin cambios. |

## Prueba de ALG-03 Generar lista de alertas

| Conjunto | Datos de entrada | Resultado esperado |
|---|---|---|
| 1 (con alertas) | FIL-001: stock 10, mínimo 4; BAL-010: stock 2, mínimo 2; ACE-020: stock 1, mínimo 3 | lista = [ACE-020, BAL-010] ordenada por stock; se muestran en rojo (criterio 1 de HU-03). |
| 2 (sin alertas) | FIL-001: stock 10, mínimo 4; BAL-010: stock 8, mínimo 2 (después de reabastecer) | lista vacía; se muestra «No hay refacciones en stock mínimo» (criterio 2 de HU-03). |

**Traza del conjunto 1 (válido):**

| Paso | i | stock | stockMinimo | Condición evaluada | lista / Salida |
|---|---|---|---|---|---|
| 1 | — | — | — | lista <- vacía; n = 3 | lista = [] |
| 2 | 1 | 10 | 4 | 10 <= 4: Falso | FIL-001 sin alerta |
| 3 | 2 | 2 | 2 | 2 <= 2: Verdadero | Agrega BAL-010; alerta = Verdadero |
| 4 | 3 | 1 | 3 | 1 <= 3: Verdadero | Agrega ACE-020; alerta = Verdadero |
| 5 | — | — | — | lista vacía: Falso; ordenar por stock ascendente | [ACE-020, BAL-010] en rojo |

**Traza del conjunto 2 (error):**

| Paso | i | stock | stockMinimo | Condición evaluada | lista / Salida |
|---|---|---|---|---|---|
| 1 | — | — | — | lista <- vacía; n = 2 | lista = [] |
| 2 | 1 | 10 | 4 | 10 <= 4: Falso | FIL-001 sin alerta |
| 3 | 2 | 8 | 2 | 8 <= 2: Falso | BAL-010 sin alerta |
| 4 | — | — | — | lista está vacía: Verdadero | No hay refacciones en stock mínimo |
| 5 | — | — | — | — | lista = [] |

## Prueba de ALG-04 Iniciar sesión

| Conjunto | Datos de entrada | Resultado esperado |
|---|---|---|

| Conjunto | Datos de entrada | Resultado esperado |
|---|---|---|
| 1 (válido) | Intento 1: usuario = admin, contraseña correcta | acceso = Verdadero en el primer intento; se abre la sesión y se muestra el panel principal. |
| 2 (error) | Intentos 1, 2 y 3: usuario = admin con contraseña incorrecta | intentos llega a 3; se muestra tres veces «Usuario o contraseña incorrectos» y luego «Cuenta bloqueada 5 minutos». |

**Traza del conjunto 1 (válido):**

| Paso | intentos | acceso | Condición evaluada | Salida |
|---|---|---|---|---|
| 1 | 0 | Falso | intentos <- 0; acceso <- Falso | — |
| 2 | 0 | Falso | 0 < 3 y acceso = Falso: Verdadero; se leen credenciales | — |
| 3 | 0 | Verdadero | registro existe y verificarHash(contraseña): Verdadero | — |
| 4 | 0 | Verdadero | 0 < 3 y acceso = Falso: Falso | Sale del ciclo |
| 5 | 0 | Verdadero | acceso = Verdadero | Se abre la sesión y se muestra el panel principal. |

**Traza del conjunto 2 (error):**

| Paso | intentos | acceso | Condición evaluada | Salida |
|---|---|---|---|---|
| 1 | 0 | Falso | intentos <- 0; acceso <- Falso | — |
| 2 | 1 | Falso | 0 < 3 y acceso = Falso: Verdadero; verificarHash: Falso; intentos <- 1 | Usuario o contraseña incorrectos |
| 3 | 2 | Falso | 1 < 3 y acceso = Falso: Verdadero; verificarHash: Falso; intentos <- 2 | Usuario o contraseña incorrectos |
| 4 | 3 | Falso | 2 < 3 y acceso = Falso: Verdadero; verificarHash: Falso; intentos <- 3 | Usuario o contraseña incorrectos |
| 5 | 3 | Falso | 3 < 3: Falso (sale del ciclo); acceso = Verdadero: Falso; bloquear(admin, 5 min) | Cuenta bloqueada 5 minutos |

## Diagramas de flujo

Cada algoritmo se representó con la simbología ANSI/ISO 5807: óvalo para inicio y fin, rectángulo para procesos, rombo para decisiones, paralelogramo para entradas y salidas, y flechas para el flujo. Los diagramas se revisaron paso a paso contra el pseudocódigo; los archivos fuente están en /docs/diseno del repositorio.

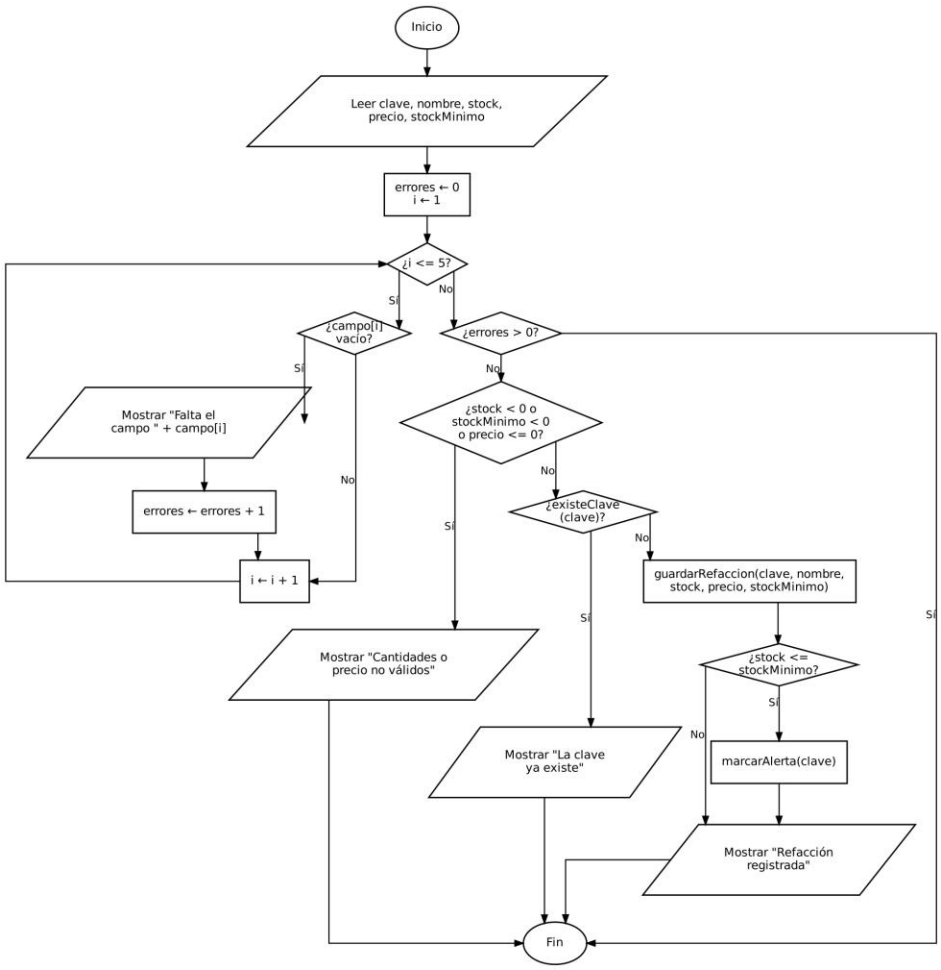



*Ilustración 2 Diagrama de flujo de ALG-01 Registrar refacción*

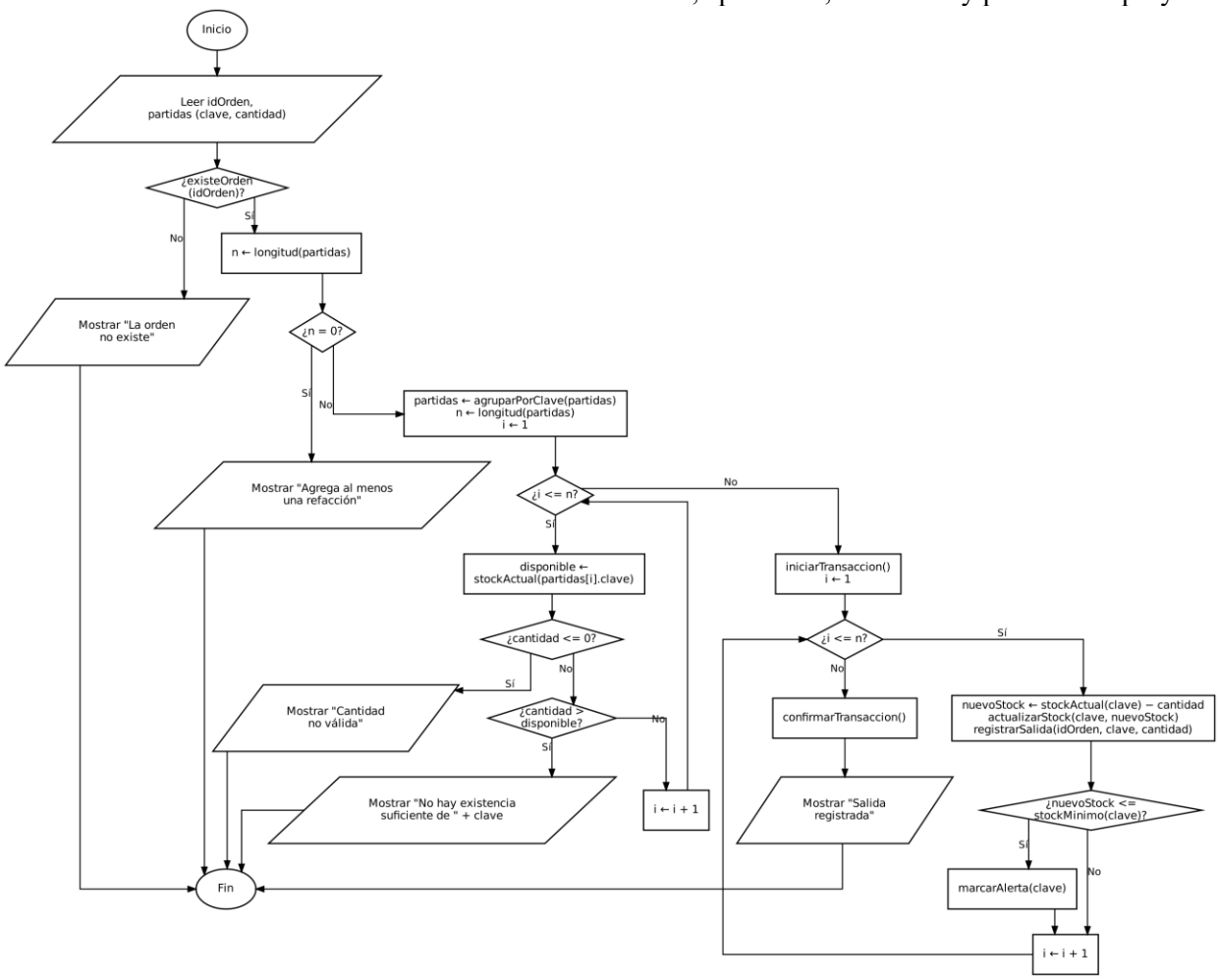



*Ilustración 3 Diagrama de flujo de ALG-02 Descontar refacciones de una orden*

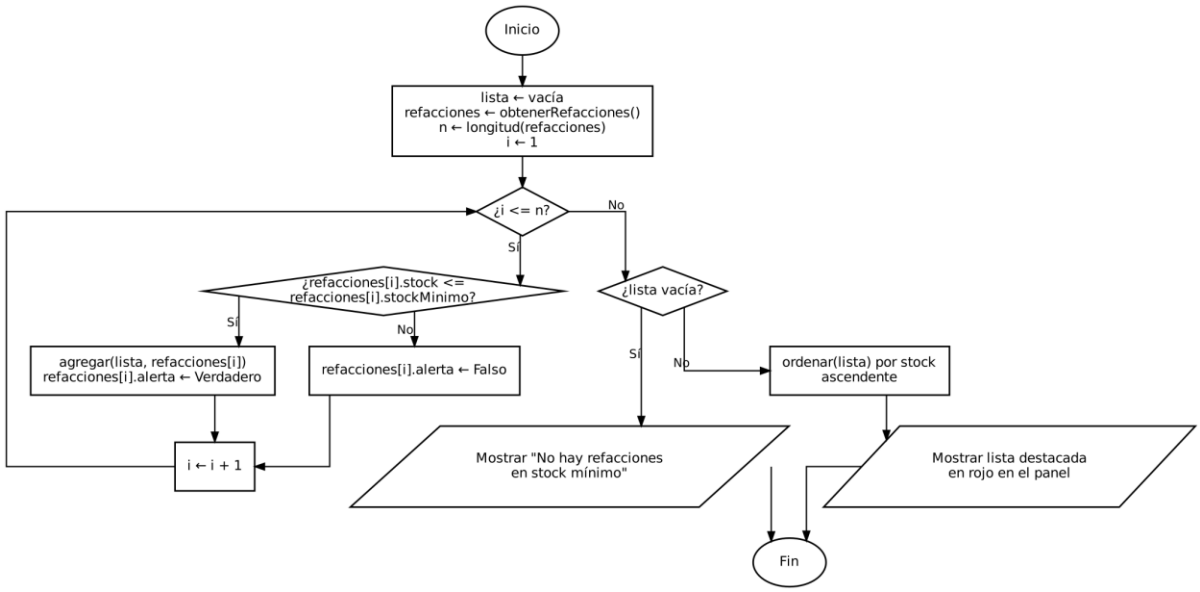



*Ilustración 4 Diagrama de flujo de ALG-03 Generar lista de alertas*

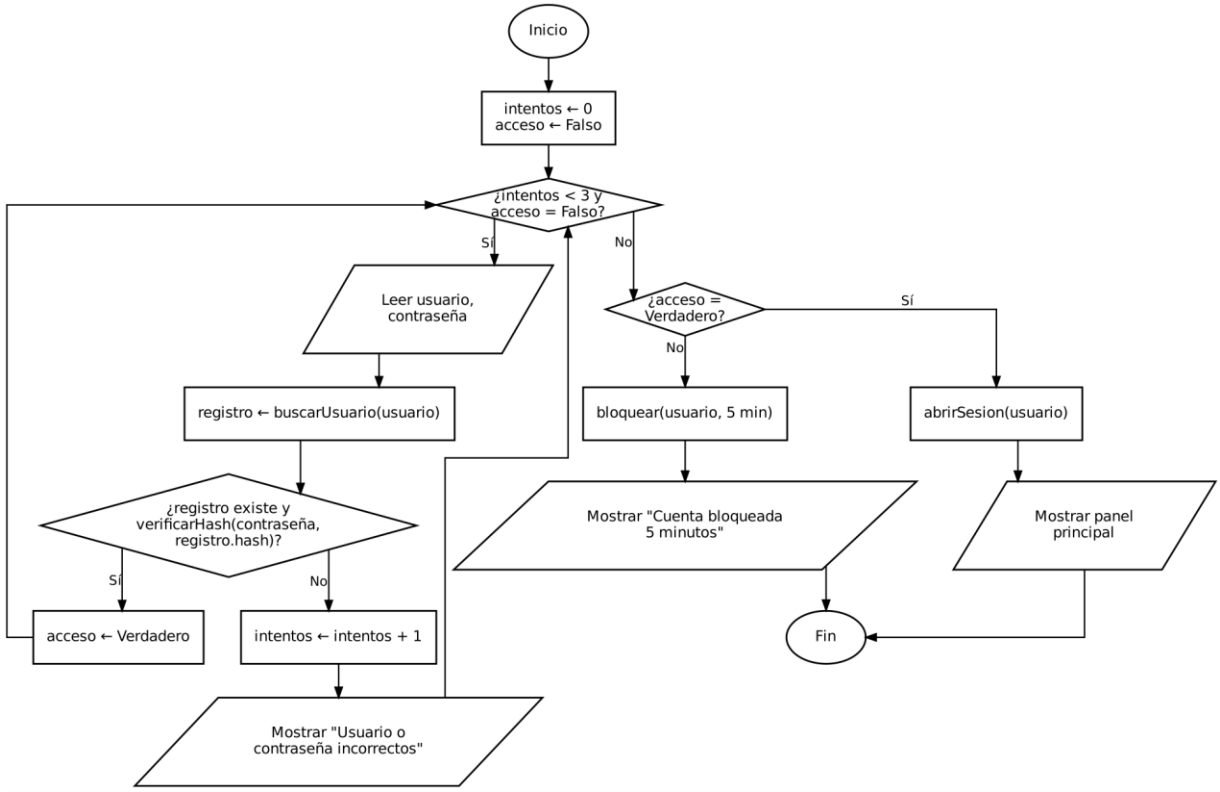



*Ilustración 5 Diagrama de flujo de ALG-04 Iniciar sesión*

## Módulos y principios de diseño funcional

El sistema se divide en tres capas para separar responsabilidades: presentación (pantallas web en Python), lógica de negocio (los módulos de la tabla) y acceso a datos (consultas a PostgreSQL). Ningún módulo de lógica escribe SQL directamente ni dibuja pantallas. Por indicación del docente, la base de datos cambió de MySQL (A05) a PostgreSQL; como el diseño funcional no depende del motor, solo cambia la capa de acceso a datos.

| Módulo | Responsabilidad única | Depende de | Principio aplicado y justificación |
|---|---|---|---|
| **Autenticación** | Validar las credenciales del administrador, controlar intentos y mantener la sesión. | Acceso a datos | Alta cohesión: solo maneja la sesión. Los demás módulos solo preguntan si hay sesión activa, no revisan contraseñas. |
| **Inventario** | Alta, edición, consulta y reabastecimiento de refacciones. | Acceso a datos, Alertas | Modularidad: concentra todas las reglas del catálogo (RN-01 a RN-03). |
| **Órdenes de servicio** | Registrar órdenes y sus salidas de refacciones con descuento transaccional. | Inventario, Clientes, Acceso a datos | Bajo acoplamiento: pide al módulo Inventario validar y descontar, en lugar de modificar el stock por su cuenta. |

| Módulo | Responsabilidad única | Depende de | Principio aplicado y justificación |
|---|---|---|---|
| **Alertas** | Calcular qué refacciones están en stock mínimo y armar la lista del panel. | Inventario (solo lectura) | Alta cohesión: una sola regla (RN-06) en un solo lugar; Inventario y Órdenes lo llaman tras cambiar el stock. |
| **Clientes y vehículos** | Registrar clientes y vehículos, buscarlos y mostrar su historial. | Acceso a datos, Órdenes (lectura del historial) | Modularidad: separa la atención al cliente del control de almacén. |
| **Acceso a datos** | Ejecutar las consultas SQL y las transacciones contra PostgreSQL. | PostgreSQL | Separación de responsabilidades: si cambia el motor o una tabla, solo se modifica esta capa. |
| **Presentación** | Mostrar formularios, listas y mensajes. | Módulos de lógica | Separación de responsabilidades: la interfaz no contiene reglas de negocio, solo las muestra. |

## *Matriz de trazabilidad*

La matriz conecta cada requisito de A05 con su historia de usuario, su caso de uso y su algoritmo o diagrama. Los huecos se marcan y se explican en la columna de observaciones.

| Requisito | Historia de usuario | Caso de uso | Algoritmo / diagrama | Observaciones |
|---|---|---|---|---|
| **RF-01** | HU-01 (Must) | CU-02 | ALG-01 / Ilustración 2 | Completo. La edición reutiliza las validaciones de ALG-01. |
| **RF-02** | HU-02 (Must) | CU-03, CU-04 | ALG-02 / Ilustración 3 | Completo. |
| **RF-03** | HU-03 (Should) | CU-05, CU-06, CU-09 | ALG-03 / Ilustración 4 | Hueco parcial: CU-09 Reabastecer no tiene algoritmo propio; es la operación inversa de ALG-02 (suma en lugar de resta) y se detallará en el Sprint 2. |
| **RF-04** | HU-04 (Should) | CU-07, CU-08, CU-10 | — | Hueco: no hay algoritmo porque el registro y la búsqueda son operaciones directas de alta y consulta SQL con las validaciones RN-10 y RN-11. Se diseñará en el Sprint 2, junto con su historia. |
| **RNF-02** | — (se verifica en la DoD) | CU-01 | ALG-04 / Ilustración 5 | Hueco: el requisito no funcional no tiene historia propia en A05. Además, RNF-02 pedía roles (administradora y mecánico) y ahora hay un solo usuario; se propone actualizar RNF-02 y agregar al Product Backlog la historia «Como administrador quiero iniciar sesión…». |

## *Revisión entre pares*

El caso de uso CU-03 y el algoritmo ALG-02 se intercambiaron con el equipo _Bendita Integración_____. La tabla registra los hallazgos que encontró el otro equipo y la corrección que se aplicó.

| # | Hallazgo del otro equipo | Elemento afectado | Corrección aplicada |
|---|---|---|---|
| 1 | No se especificaba qué sucede cuando se intenta agregar una refacción que no existe en el catálogo. | CU-03 | Se agregó una validación para comprobar que la refacción esté registrada antes de continuar con el descuento. |
| 2 | El algoritmo no especificaba que las cantidades de refacciones debían ser números enteros positivos. | ALG-02 | Se agregó una validación para rechazar cantidades negativas, iguales a cero o con valores decimales. |
| 3 | La validación del stock se realizaba antes de iniciar la transacción, lo que podía generar inconsistencias si el inventario cambiaba durante el proceso. | ALG-02 | Se ajustó el orden de las operaciones para realizar las validaciones dentro de la transacción y mantener la integridad del inventario. |

## Conclusiones

Con esta actividad el equipo representó la lógica de negocio del Sistema de Gestión de Clientes Mqueen antes de programar. El diagrama de casos de uso delimita qué hace cada actor, y las especificaciones de CU-02 y CU-03 dejan por escrito los flujos de error que piden los criterios de aceptación de HU-01 y HU-02: no guardar si falta un campo y no descontar si no hay existencia suficiente.

Los cuatro algoritmos y sus diagramas de flujo serán la guía directa del código en los Sprints 1 y 2. El diseño de ALG-02, que valida todo antes de descontar y usa una transacción, ataca la causa del problema de A02: el 70 % de las refacciones no coincidía con el registro. La división en módulos con bajo acoplamiento permite repartir el trabajo entre los integrantes, y la matriz de trazabilidad muestra qué falta diseñar: el reabastecimiento, el módulo de clientes y una historia para el inicio de sesión, que se atenderán en el Sprint 2.

### *Declaración de uso de IA*

Se utilizó Claude (Anthropic) para dar formato al informe, para revisar la notación UML del diagrama de casos de uso y para redactar borradores de las especificaciones de casos de uso, del catálogo de reglas de negocio, de los algoritmos en pseudocódigo y de la tabla de módulos, todos a partir de las historias de usuario y requisitos de A05. También se usó para generar las imágenes de los diagramas, para verificar que coincidieran con el pseudocódigo y para revisar y corregir las trazas de las pruebas de escritorio que llenó el equipo.

# Bibliografía

Clark, W. (2020). Metodología ágil: Una guía para principiantes sobre el método y los principios ágiles. Independently Published.

Hernández Sampieri, R. y Mendoza Torres, C. P. (2023). Metodología de la investigación. McGraw Hill.

International Organization for Standardization. (1985). ISO 5807:1985 Information processing — Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts. ISO.

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML) Version 2.5.1. https://www.omg.org/spec/UML/2.5.1

Herrera González, R. I. (2026). Manual de Proyecto Integrador I. Universidad Tecnológica de Durango.